MODULE 20 · SECTION 3 - STORING THE DATA

Bigtable: Row Keys and Performance

Row key design, hotspots, tall and wide tables, garbage collection, storage, scaling and the newest features.

SECTION 3 of 5 EXAM WEIGHT 20% MODULE 20 of 45

admin@sthithapragna.com

A row key is a phone book. Everyone called Smith on one page is a hotspot.

One sorted key, split across nodes

- Rows are **sorted by row key**, as bytes, and split into ranges called tablets.
- Each node serves some tablets. Bigtable **rebalances** them as load changes.
- Fast reads: **one key**, a **prefix**, or a **key range**.
- There is **one index**, the row key. Anything else means scanning.

> THE RULE
>
> Design the key around the questions you will ask. You cannot add another index later.

Larkspur's two row keys

THE PROTOTYPE

reading_ts#meter_id

- Every new reading has the newest time
- So every write lands at the end
- One tablet, one node, red hot
- Other nodes sit idle

THE FIX

meter_id#reverse_ts

- 3.1 million different prefixes
- Writes spread across every node
- Each meter's newest reading first
- One prefix scan per meter

THE PATTERN

Time first makes a hotspot. Identity first, then time, spreads the load and keeps a meter's rows together.

Row key patterns to know

PATTERN	WHAT IT DOES	WATCH OUT
Identity first	Spreads writes across the key space	Pick a high-cardinality field
Reverse timestamp	Newest rows sort first	Never at the very start
Field promotion	Moves a queried field into the key	Order fields by how you filter
Salting	A small prefix bucket spreads hot writes	Reads must fan out to every bucket
Hashing the key	Even spread	Loses order; Google advises readable keys
Reverse domain	com.company.product groups by site	Only helps with many domains

THE TIEBREAKER

Readable, high-cardinality prefixes beat clever tricks. Salt only when one value is unavoidably hot.

Building Larkspur's row key

```python
MAX_LONG = 2**63 - 1

def row_key(meter_id: str, reading_ts_ms: int) -> bytes:
    reverse_ts = MAX_LONG - reading_ts_ms
    return f"{meter_id}#{reverse_ts:019d}".encode()

# Bad:  2026-10-09T17:30Z#MTR00012345   every write lands at the end
# Good: MTR00012345#9223370245287775807  spread by meter, newest first

# Latest reading: scan the prefix MTR00012345#, limit 1 row
```

4 Subtracting from the maximum long makes newer times sort first.

5 Padded to a fixed width, so the bytes sort correctly.

REMEMBER

Keep each row key well under the 4 KB limit. Short keys are cheaper to store and compare.

Tall or wide?

TALL

One row per reading

- Key: meter#reverse_ts
- Small rows, simple writes
- Garbage collect by age
- Google's default advice for time series

WIDE

One row per meter per day

- Key: meter#date, 48 columns
- Fewer rows, one read per day
- Row-level atomic updates
- Keep rows under 100 MB

THE NUMBERS

Limits: key 4 KB, cell 10 MB, row ideally under 100 MB and never over 256 MB, about 100 column families.

Column families and garbage collection

- Each **column family** has its own **garbage collection** policy.
- Rules: **max versions**, **max age**, combined as union or intersection.
- **No default TTL**. gcloud and cbt keep unlimited versions unless you say otherwise.
- Deletion happens at compaction, **up to a week** later. Filter reads to match.

> FOR LARKSPUR
>
> Put data with different lifetimes in different families: raw for 90 days, summaries kept for years.

Key Visualizer: see the hotspot

- A **heatmap**: time across, row-key ranges down. **Bright** means hot.
- A bright **horizontal band** is a hot key range.
- A steady **diagonal** marks sequential keys, such as timestamps first.
- Needs at least **1 GB per cluster**. New tables take days to show data.

> THE SIGNAL
>
> 'Some nodes busy, others idle' plus a bright band in Key Visualizer means fix the row key, not the node count.

SSD or HDD?

SSD

The default for almost everything

- 5 TB per node
- Low, predictable latency
- Serving and real-time reads
- Usually the cheapest overall

HDD

Only for patient batch data

- 16 TB per node
- 5% of SSD's read rows per second
- Scans and writes, no users waiting
- Can need more nodes than it saves

THE TRAP

Storage type is permanent. To change it, back up the table and restore it into a new instance.

Nodes and autoscaling

- Each node serves roughly **17,000 requests a second**, by Google's figure.
- Autoscaling targets **CPU**, from 10% to 80%, and **storage**, 50% by default.
- Scales up **immediately**, down slowly. Rebalancing takes minutes.
- Latency-sensitive apps: keep storage utilisation **under 60%**.

> THE TRAP
>
> More nodes help only when load is spread. A hotspot stays a hotspot on a bigger cluster.

Editions and newer features

- **Editions, since April 2026**: Enterprise, and Enterprise Plus. Older instances are Enterprise.
- **Enterprise Plus**: an in-memory tier for sub-millisecond reads (Preview), bigger tiered storage.
- **Data Boost**: separate compute for batch and analytical reads, so serving is not disturbed.
- **Continuous materialized views**: SQL results kept up to date, even with a different key.

> THE SIGNAL
>
> 'Analytics on Bigtable without hurting the app' points to Data Boost.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Some nodes busy, others idle	Redesign the row key
Key starts with a timestamp	Identity first, reverse timestamp after
Newest first for each device	Reverse timestamp in the key
Old cells still returned	Garbage collection rule, plus a read filter
Batch only, no users, huge volume	HDD, in a new instance
Analytics must not hurt serving	Data Boost

THE PATTERN

The row key decides performance. Nodes, disks and editions only tune it.

> Larkspur's readings table uses the row key reading_ts#meter_id. During the evening surge, one node runs at full CPU while the others are nearly idle.
>
> Key Visualizer shows one bright band at the end of the key range.

What should the team change?

A Double the node count, so that the surge has more capacity

B Move the table to HDD storage, which holds more data per node

C Hash the entire row key, so that every write lands at random

D Lead the key with the meter ID, then add a reversed timestamp

◆ PAUSE AND DECIDE

What should the team change?

A Double the nodes

The hot range still lives on one node. The others stay idle.

B Move to HDD

Slower reads, and no change to where writes land.

C Hash the whole key

Spreads writes, but per-meter range reads become impossible.

D Meter first, then time

Writes spread across meters, and each meter's newest row is first.

THE SIGNAL

One node hot, a bright band at the end, timestamp first: redesign the key, identity first.

> Larkspur keeps raw readings in a Bigtable column family created with cbt. Policy says raw cells must be kept for 90 days only.
>
> The table keeps growing, and some queries still return cells older than 90 days.

What should the team do?

A Set a max-age rule of 90 days on the family, and filter reads to match

B Write a nightly job that scans the whole table and deletes the old rows

C Set a max-versions rule of one cell on the column family

D Set a 90-day time to live on the table, instead of the family

◆ PAUSE AND DECIDE

What should the team do?

A Max age, plus a filter

Garbage collection removes old cells; the filter hides them meanwhile.

B A nightly delete job

A full scan every night, doing what garbage collection does free.

C Max versions of one

Keeps the newest cell per column, not 90 days of readings.

D A table-level TTL

Garbage collection rules are set per column family.

THE SIGNAL

Keep for N days in Bigtable: a max-age rule per family, plus a matching read filter.

A 400 TB Bigtable instance on SSD holds years of grid sensor data. It is read only by a daily batch job that scans it for reports.

No application or person waits on it, and finance wants the storage bill cut.

What should the team do?

A Switch the existing instance's storage type from SSD to HDD

B Create an HDD instance, and restore the table there from a backup

C Keep SSD, and add nodes so that the daily scans finish faster

D Move the sensor data into Memorystore, to make reads cheaper

◆ PAUSE AND DECIDE

What should the team do?

A Switch in place

Storage type is permanent. It cannot be changed on an instance.

B New HDD instance, restore

Batch-only scans suit HDD, and backup and restore is the move.

C More SSD nodes

Raises the bill, which is the opposite of the goal.

D Memorystore

Memory is the most expensive place to keep 400 TB.

THE SIGNAL

Batch-only, latency-insensitive, huge: HDD. But storage type is permanent, so restore into a new instance.

> Larkspur's analysts run large daily scans of the live readings table for reports. While they run, the customer app's read latency spikes.
>
> The app must stay fast, and the analysts still need their scans every day.

What should the team do?

A Raise the autoscaling CPU target, so that the existing nodes work harder

B Switch the instance to HDD, so that big scans are cheaper

C Run the analysts' scans through Data Boost, apart from serving

D Copy the whole table into Memorystore, just for the analysts

◆ PAUSE AND DECIDE

What should the team do?

A Higher CPU target

Fewer nodes, hotter nodes, and worse latency for the app.

B Switch to HDD

Slower reads for the app, and storage type is permanent.

C Data Boost

Separate compute for the scans, so serving nodes are untouched.

D Memorystore copy

An expensive copy of a huge table, to fix a scheduling clash.

THE SIGNAL

Analytics must not hurt serving: Data Boost, or a separate cluster and app profile.

Module 20 in five lines

- One index, the **row key**. Design it around the questions you will ask.
- Never lead with a timestamp. **Identity first**, then a **reverse timestamp**.
- Garbage collection is **per family**, with no default TTL. Filter reads to match.
- **SSD** by default. **HDD** only for batch scans, and the choice is permanent.
- Hotspots need a new key, not more nodes. **Data Boost** isolates analytics.

COMING UP

Next: module 21, relational and operational stores.